



NICIS

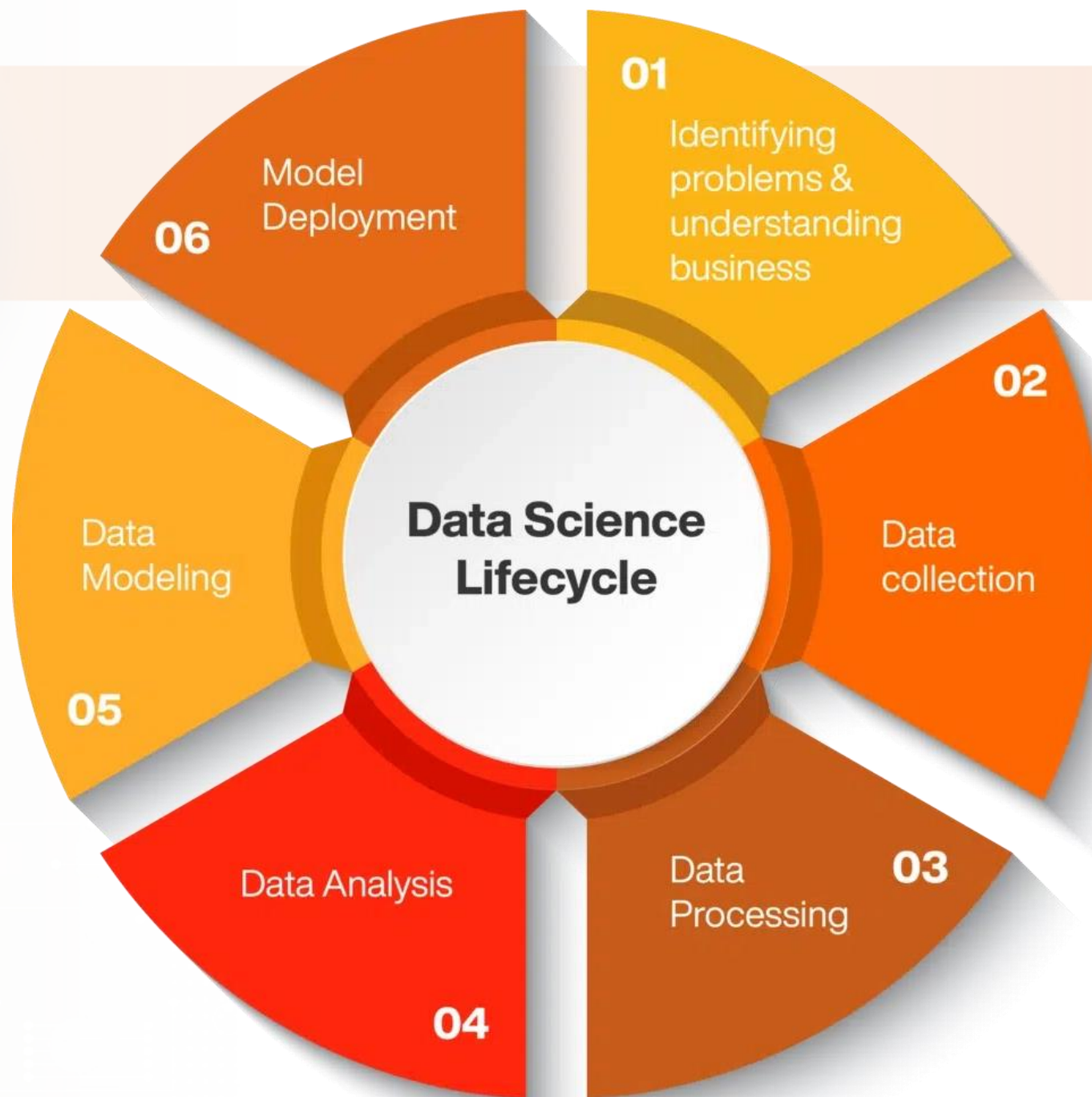
NATIONAL INTEGRATED
CYBERINFRASTRUCTURE SYSTEM

DIRISA

Introduction to Data Exploration and Preprocessing

Miss Boitshepo Sebusho (Data Scientist)

07 July 2026





Introduction

After a problem is identified and

Defined,

The data acquired and

Read into the appropriate platform (e.g. Google Colab, Pycharm, Jupyter notebook.

Next the data should be

Checked for quality

Explored for better understanding,

Cleaned and processed if needed*



01

Data Exploration



Data Exploration

What is Data Exploration?

- Data exploration is the process of examining a dataset to understand its characteristics, quality, and patterns before analysis or modelling.
- An important step in any data analysis or machine learning project
- Involves understanding the structure, patterns, and anomalies in Data
- Preliminary investigation to get a feel of the data and prepare it for further steps

Raw Data

Data Exploration

Data Preprocessing

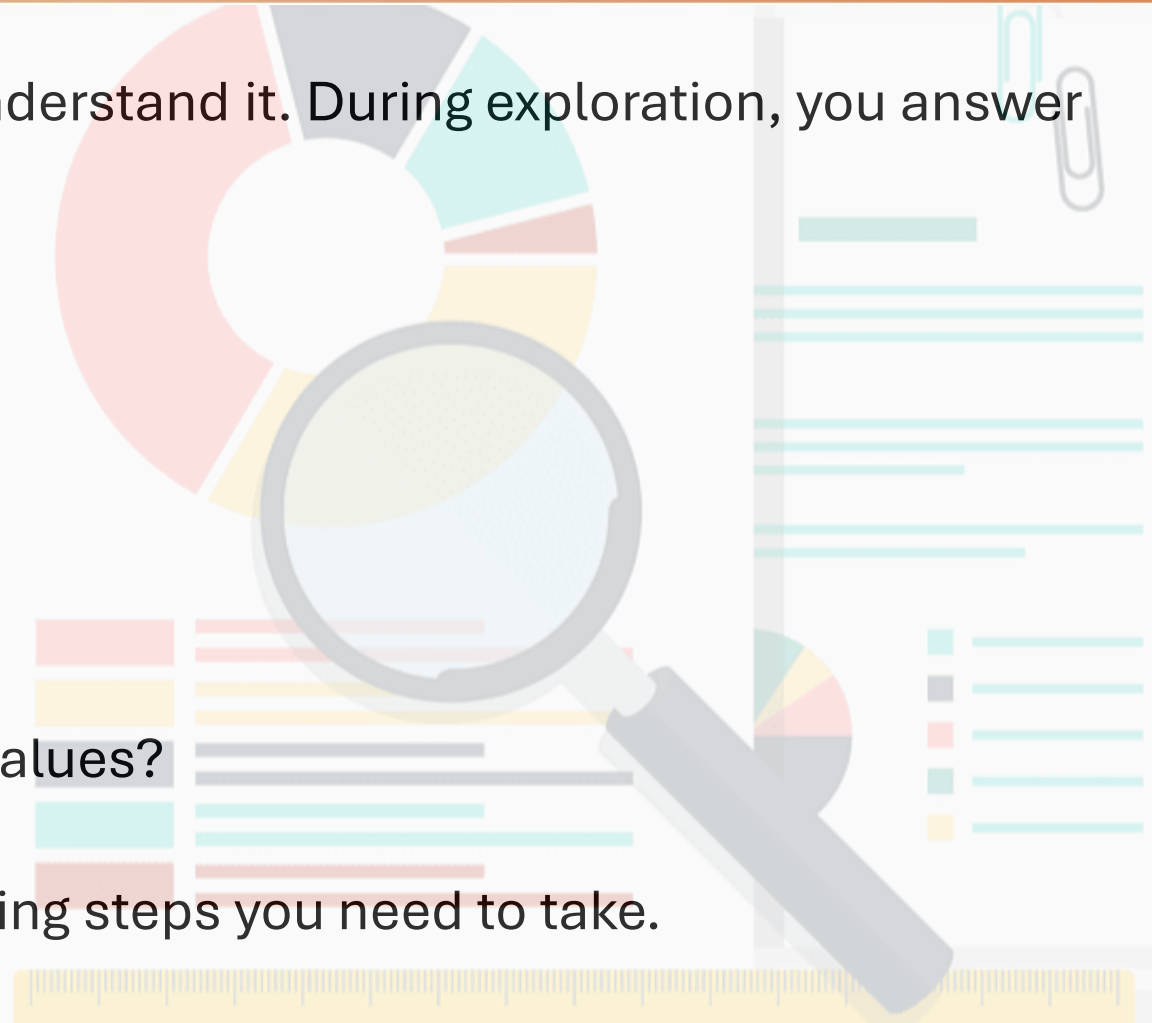
Modelling/ Analysis



Data Exploration

You can't clean data effectively until you understand it. During exploration, you answer questions such as:

- What variables are in the dataset?
- What are the data types?
- How many rows and columns are there?
- Are there missing values?
- Are there duplicate records?
- Are there outliers?
- Are there inconsistencies in categorical values?
- What do the distributions look like?
- These findings determine the preprocessing steps you need to take.





Data Exploration

Goals:

- Understand data types and distributions
- Identify missing values or outliers
- Detect patterns or trends
- Generate initial hypotheses
- Select features for modeling

This data is going to be used to make business decisions.



Understanding the Dataset

By Structure	Structured Data	Unstructured Data
Description	Clearly defined fields, rows, and columns (e.g., spreadsheets, SQL databases).	No fixed format; may include text, images, audio, video.
Example	Sales records, customer tables.	Emails, social media posts, YouTube videos.
Tools	Excel, SQL, Pandas.	NLP libraries, CV models

By Source	Primary Data	Secondary Data
Description	Collected firsthand through experiments, surveys, or sensors.	Previously collected and available through other sources.
Example	Survey responses, lab experiment results.	Open government data, public datasets (e.g., Kaggle, UCI).



Understanding the Dataset

By Time Component	Cross-Sectional Data	Time Series Data	Panel (Longitudinal) Data
Description	Collected at a single point in time.	Data collected over time (daily, monthly, yearly).	Multiple observations over time for the same subjects.
Example	Census data, customer data snapshot.	Stock prices, weather logs.	Tracking patients' health over several years.



Understanding the Dataset

What you want to know??

- Number of rows and columns
- Features (variables)
- Observations (records)
- Data types
- Units of measurement
- Target variable (if applicable)





Initial Data Examination

If data is in pandas DataFrame:

- Read the file as a dataframe
- Preview the first few rows of the DataFrame: **df.head()**

[14]
✓ 0s

```
import pandas as pd
import seaborn as sns

# Load the iris dataset
iris = sns.load_dataset('iris')
```

iris.head()						
▼	...	sepal_length	sepal_width	petal_length	petal_width	species
0		5.1	3.5	1.4	0.2	setosa
1		4.9	3.0	1.4	0.2	setosa
2		4.7	3.2	1.3	0.2	setosa
3		4.6	3.1	1.5	0.2	setosa
4		5.0	3.6	1.4	0.2	setosa



Initial Data Examination

df.info() - Concise summary of the DataFrame:

- The names of each column.
- The number of non-null values in each column, which helps identify missing data.
- The data type (dtype) of each column.

df.shape - Return a tuple representing the shape of the DataFrame. The shape is the number of rows and columns of the DataFrame:

```
[15] iris.info()  
✓ 0s  
... <class 'pandas.core.frame.DataFrame'>  
RangeIndex: 150 entries, 0 to 149  
Data columns (total 5 columns):  
#   Column          Non-Null Count  Dtype  
---  ---  
0   sepal_length    150 non-null   float64  
1   sepal_width     150 non-null   float64  
2   petal_length    150 non-null   float64  
3   petal_width     150 non-null   float64  
4   species         150 non-null   object  
dtypes: float64(4), object(1)  
memory usage: 6.0+ KB  
  
[16] iris.shape  
✓ 0s  
... (150, 5)
```



Initial Data Examination

Descriptive statistics: **df.describe()**

For numeric columns:

- count: Number of non-null observations.
- mean: The average value.
- std: Standard deviation, a measure of data dispersion.
- min: Minimum value.
- 25%: The 25th percentile (first quartile, Q1).
- 50%: The 50th percentile (median, Q2).
- 75%: The 75th percentile (third quartile, Q3).
- max: Maximum value.

A screenshot of a Jupyter Notebook interface. The top part shows the code `iris.describe()` in a cell. Below the code, the output is displayed as a table with 5 columns: the statistical measure and the four numeric features of the iris dataset. The table has 9 rows, one for each statistic (count, mean, std, min, 25%, 50%, 75%, max) and one for each feature (sepal_length, sepal_width, petal_length, petal_width).

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

Initial Data Examination

Generates descriptive statistics for a specific column in a Dataframe:

`df['column_name'].describe()`

- Returns all unique values in a column:

`df['column_name'].unique()`

- Returns counts of unique values in a column:

`df['column_name'].value_counts()`

```
iris['species'].describe()
```

```
..          species
count         150
unique           3
top         setosa
freq           50
```

```
dtype: object
```

```
iris['species'].unique()
```

```
array(['setosa', 'versicolor', 'virginica'], dtype=object)
```

```
iris['species'].value_counts()
```

```
..          count
species
setosa         50
versicolor     50
virginica      50
```

```
dtype: int64
```



Initial Data Examination

df.iloc[] is used for integer-location based indexing and

allows you to select data based on their numerical position starting from 0

df.iloc[start:end] returns rows between a specified range using slicing

```
iris.iloc[9]
```

	9
sepal_length	4.9
sepal_width	3.1
petal_length	1.5
petal_width	0.1
species	setosa

dtype: object

```
iris.iloc[9:14]
```

	sepal_length	sepal_width	petal_length	petal_width	species
9	4.9	3.1	1.5	0.1	setosa
10	5.4	3.7	1.5	0.2	setosa
11	4.8	3.4	1.6	0.2	setosa
12	4.8	3.0	1.4	0.1	setosa
13	4.3	3.0	1.1	0.1	setosa



02

Data Preprocessing



Data Preprocessing

What is Preprocessing?

- The process of cleaning and transforming raw data to a usable format.
- By processing raw data, we turn it into useful information
- Prepares data for modeling and analysis

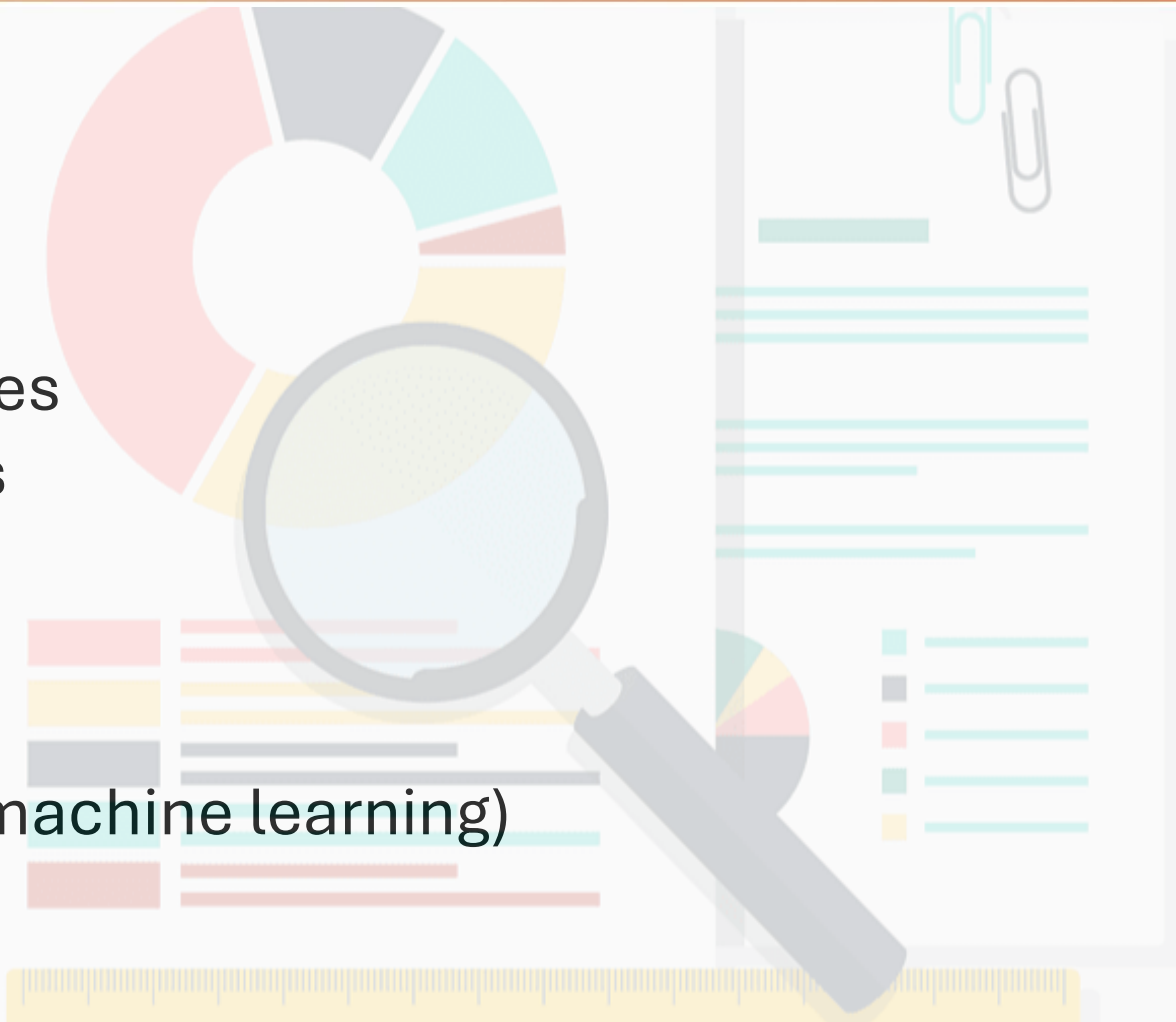
Importance of Data Preprocessing:

- Real world data is often messy
- Models require numerical and clean input
- Better preprocessing → better model performance
- Overall, we do this to get meaningful insights for informed decision-making



Key Data Preprocessing Steps

- Handling missing values
- Removing duplicates
- Correcting data types
- Standardizing categorical values
- Encoding categorical variables
- Feature scaling
- Feature engineering
- Handling outliers
- Splitting data (if preparing for machine learning)
- Validating the cleaned dataset





Data Quality Assessment: Missing Values

- Check if there are any missing values: **`df.isnull().sum()`**
- The existence of any NaN entries in the entire dataframe can be identified: **`df.isnull().values.any()`**

```
df.isnull().values.any()  
• np.True_
```

- The rows (index) containing any NaNs can also be found:

`df[df.isnull().any(axis=1)].index`

```
df[df.isnull().any(axis=1)].index  
... Index([ 290,   341,   538,   563,   696,   738,  1097,  1350,  1456,  
          1493,  
          ...  
          19932, 19959, 20046, 20069, 20125, 20267, 20268, 20372, 20460,  
          20484],  
          dtype='int64', length=207)
```

Handling Missing Values

```
df.isnull().sum() #checking for missing values
```

	0
longitude	0
latitude	0
housing_median_age	0
total_rooms	0
total_bedrooms	207
population	0
households	0
median_income	0
median_house_value	0
ocean_proximity	0

dtype: int64

Only one column has missing values.

🧠 Think: How should we deal with this? Is it OK to fill? Should we drop?

Option 1: Drop

- ✓ Quick and simple.
- ⚠ Risk: You lose 207 rows. Only use if data loss is acceptable.

Option 2: Fill with Median or Mean

- ✓ Keeps all rows.
- 🔍 Median is more robust than mean if outliers exist.



Handling Missing Values

Choosing the Best Method

Before deciding, ask:

- How much data is missing?
- Why is it missing?
- Are there outliers?
- Is the column important?

Situation	Recommended Action
Small amount of missing data	Drop rows or fill values
Large amount of missing data	Fill (impute) values
Data has outliers	Use Median
Data has no outliers	Use Mean
Column has too many missing values (e.g., >50%)	Consider dropping the column



Handling Missing Values

1. Measure how much is missing

Rule of thumb:

Missing %	Common Action
Less than 5%	Drop or fill
5–20%	Usually fill (impute)
More than 20%	Investigate carefully
More than 50%	Consider dropping the column

2. Understand why values are missing

Ask:

- Did people skip a question?
- Was data not recorded?
- Is the missingness random?

Sometimes missing values contain important information.



Handling Missing Values

3. Check the data distribution

Look for outliers.

- If there are no strong outliers → Mean is often fine.
- If there are outliers → Median is usually better.

Example

Income data often has extreme values, so median is preferred.

4. Consider the importance of the column

Ask:

- Is this column important for predictions or analysis?
- Would dropping rows remove useful information?

If the column is important, filling is often better than dropping many rows.



Handling Missing Values (Impute)

Fill with Median or Mean:

- Keeps all rows.
- Median is more robust than mean if outliers exist.

```
2] median_bedrooms = df['total_bedrooms'].median()  
0s df['total_bedrooms'] = df['total_bedrooms'].fillna(median_bedrooms)
```

```
] df.isnull().sum()
```

	0
longitude	0
latitude	0
housing_median_age	0
total_rooms	0
total_bedrooms	0
population	0

Handling Missing Values (Impute)

Using simpleImputer

```
▶ from sklearn.impute import SimpleImputer

# Create the imputer
imputer = SimpleImputer(strategy='median')

# Apply it to the column
df[['total_bedrooms']] = imputer.fit_transform(df[['total_bedrooms']])

# Check that missing values have been filled
print(df['total_bedrooms'].isnull().sum())
```

... 0



Handling Missing Values (Drop)

Drops rows with atleast one Nan: **df.dropna(inplace=True)**.

Inplace=True is used to modify the original DataFrame.

- Drops all columns with any missing values: **df.dropna(axis=1)**
- Drop rows where all elements are missing: **df.dropna(how='all')**
- Removes all the ROWS with missing values in specific column:

```
df.dropna(subset=['total_bedrooms'], inplace = True )
```



```
df.shape
```

```
... (20433, 10)
```

- **.reset_index** to reset the index of the dataframe

Outliers

Checking outliers

Df.describe()

Look for:

Very large **max** values

Very small **min** values

Mean much larger or
smaller than the
median (50%)

df.describe()

...

index	longitude	latitude	housing_median_age	total_rooms
count	20640.0	20640.0	20640.0	20640.0
mean	-119.56970445736432	35.63186143410853	28.639486434108527	2635.7630813953488
std	2.003531723502581	2.1359523974571117	12.585557612111637	2181.615251582787
min	-124.35	32.54	1.0	2.0
25%	-121.8	33.93	18.0	1447.75
50%	-118.49	34.26	29.0	2127.0
75%	-118.01	37.71	37.0	3148.0
max	-114.31	41.95	52.0	39320.0

Show 25 per page

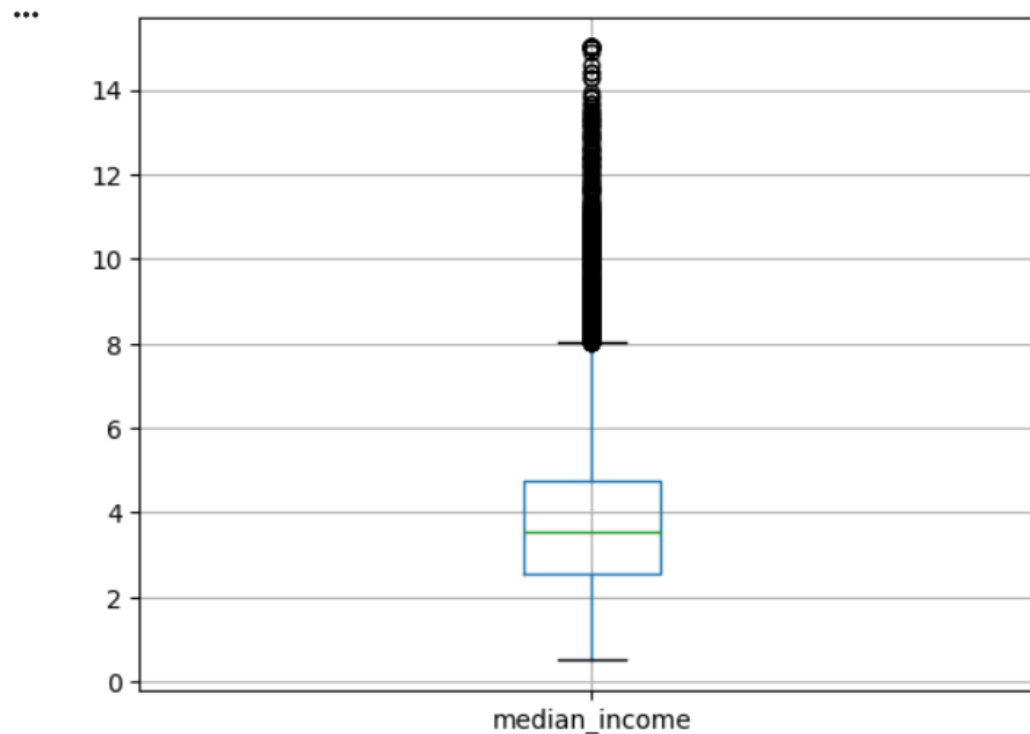
1 to 8 of 8 entries Filter ?

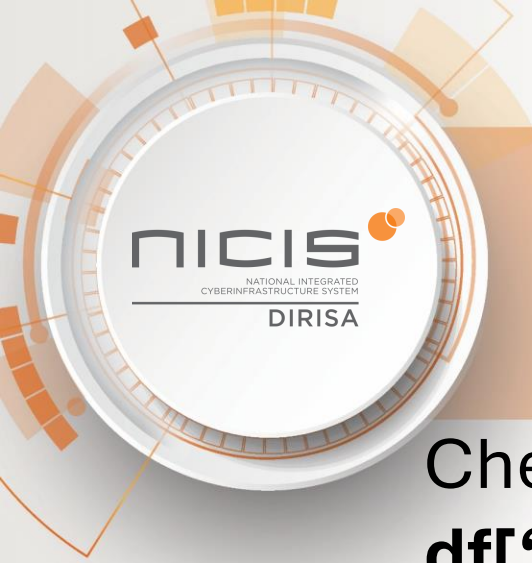
total_bedrooms	population	households	median_income	median_house_value
20433.0	20640.0	20640.0	20640.0	20640.0
537.8705525375618	1425.4767441860465	499.5396802325581	3.8706710029069766	206855.81690891474
421.3850700740322	1132.4621217653375	382.3297528316099	1.8998217179452732	115395.6158744132
1.0	3.0	1.0	0.4999	14999.0
296.0	787.0	280.0	2.5633999999999997	119600.0
435.0	1166.0	409.0	3.5347999999999997	179700.0
647.0	1725.0	605.0	4.74325	264725.0
6445.0	35682.0	6082.0	15.0001	500001.0

Detecting Outliers: Using Box Plot

```
import matplotlib.pyplot as plt

df.boxplot(column='median_income')
plt.show()
```





Detecting Outliers: Using IQR

Checking outliers by finding the quantile of a specific column:
`df['column_name'].quantile()`

```
Q1 = df['median_income'].quantile(0.25)
Q3 = df['median_income'].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = df[(df['median_income'] < lower) |
               (df['median_income'] > upper)]

print(outliers)
```

Filtering out outliers

```
df = df[~((df['median_house_value'] < (Q1 - 1.5 * IQR)) | (df['median_house_value'] > (Q3 - 1.5 * IQR)))]
```

Data Quality Assessment: Duplicates

- Check if there are any duplicate records: **df.duplicated().sum()**
- To see the duplicated row/rows: **df[df.duplicated()]**
- Removing duplicates: **df[df.isnull().any(axis=1)].index**

```
iris.duplicated().sum()
```

```
np.int64(1)
```

```
iris[iris.duplicated()]
```

	sepal_length	sepal_width	petal_length	petal_width	species
142	5.8	2.7	5.1	1.9	virginica

```
iris = iris.drop_duplicates()
```



Incorrect Data Types

Correcting Data Types

```
df["Age"] = df["Age"].astype(int)
```

```
df["Date"] = pd.to_datetime(df["Date"])
```



Handling Inconsistent Data

```
df["Gender"] = df["Gender"].replace({  
    "M":"Male",  
    "male":"Male",  
    "F":"Female",  
    "female":"Female"  
})
```





Encoding: Converting Categorical Variables to Numeric Form

Machine learning models work with numbers, not text.

- Ocean proximity is a categorical column (e.g., 'NEAR OCEAN', 'INLAND').
- We convert each category into a numeric code using pandas:
- Result:
- 'INLAND' might become 0
- 'NEAR OCEAN' → 1
- 'ISLAND' → 2, etc.

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()

df['ocean_proximity'] = encoder.fit_transform(df['ocean_proximity'])

df.head()
```

action	households	median_income	median_house_value	ocean_proximity
322	126	8.3252	452600	3
2401	1138	8.3014	358500	3
496	177	7.2574	352100	3
558	219	5.6431	341300	3
565	259	3.8462	342200	3



Feature Engineering

Examples:

Existing Features:

- Total Rooms
- Total Bedrooms
- Population
- Households

New Features:

- **Rooms per Household** = Total Rooms ÷ Households
- **Bedrooms per Room** = Total Bedrooms ÷ Total Rooms
- **Population per Household** = Population ÷ Households

```
df['rooms_per_household'] = df['total_rooms'] / df['households']  
df['bedrooms_per_room'] = df['total_bedrooms'] / df['total_rooms']  
df['population_per_household'] = df['population'] / df['households']
```



Data Processing - Randomisation

- **What is Randomization?**

- Randomly shuffles the dataset before training a machine learning model.
- Ensures the data is in a random order and helps prevent bias.

- **Why is it Important?**

- Prevents patterns caused by the original order of the data.
- Creates a fair mix of observations.
- Improves the reliability of model evaluation.

```
df = df.sample(frac=1, random_state=42).reset_index(drop=True)
```



Data Processing - Normalisation

- **What is Normalization?**
- Rescales numerical values to a **common range**, usually **0 to 1**.
- Ensures features with large values do not dominate those with smaller values.
- **Why is it Important?**
- Improves the performance of many machine learning algorithms.
- Makes features comparable.
- Helps models train faster and more effectively.



Normalisation

Feature	Value
Median Income	8.3
Total Rooms	7,099

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

df[['median_income', 'total_rooms']] = scaler.fit_transform(
    df[['median_income', 'total_rooms']]
)
```

There are built in functions for normalization
e.g., `sklearn.preprocessing.normalize(..)` and `MinMaxScaler()`.
We apply Min-Max Scaling to bring values into a range of 0–1:

After Normalization

- All values are scaled between **0 and 1**.
- Features are now on a similar scale.



03

Conclusion



Summary

- Data exploration helps you to understand your dataset
- Data Preprocessing or Data Cleaning is essential to clean and prepare your dataset
- Invest time in these steps to save effort later in modeling



Final Thoughts

- Every method that we discussed today has multiple variations. I have shown you one way to do most of the tasks in this presentation but if you were to play with it a bit or look online you will see other methods.
- The internet is a great tool – code for almost anything that you want to do now or in the future have already been posted by someone else online. Just don't give up immediately and experiment with different search terms.
- Tools such as ChatGPT can be valuable when coding (as long as you are only using it when allowed), but don't take everything provided as gospel. Sometimes additional work is needed to get the code to do exactly what is needed, and sometimes it can be way off the mark

THANK YOU



NICIS

NATIONAL INTEGRATED
CYBERINFRASTRUCTURE SYSTEM

DIRISA

Email: bsebusho@csir.co.za

